

Cuestionario teórico.

1- Sistemas Operativos, generalidades.

a- Quien es el encargado de administrar los tiempos del CPU y cuál es su función principal dentro del sistema?

En FreeRTOS, el scheduler es el encargado de administrar los tiempos del CPU. Cumple la función de administrar las tareas a ejecutarse según su prioridad, ejecutando la tarea de mayor prioridad en estado "Ready". Esto permite optimizar los tiempos de ejecución y bloqueos de tareas, según los eventos del sistema como por ejemplo colas, semáforos, mutex e interrupciones.

b- Explique a que se denomina "Contexto de Ejecución". ¿Cuáles son las variables intervinientes?

El Scheduler o planificador es el encargado de realizar los cambios de contextos al cambiar de tarea en ejecución. Ese contexto es guardado en memoria para retomar en la misma sentencia que estaba a punto de ejecutar antes del cambio, al volver a dicha tarea. Es decir, cuando una tarea no esta ejecutándose, la tarea esta inactiva y su estado ya fue salvado anteriormente. Entonces esta lista para ejecutarse y retomar desde la instrucción que estaba a punto de ejecutar.

c- ¿A qué se denomina Sistema Operativo de Tiempo Real? ¿Por qué usar un Sistema Operativo de Tiempo Real? (Pag.95)

Existen diferentes tipos de sistemas de tiempo real. Un sistema operativo de tiempo real Duro es aquel cuyo comportamiento tiene que responder dentro de tiempos límites estrictamente definidos. Si falla o demora mas tiempo, esto se traduce en una falla del sistema. En cambio, un sistema de tiempo real Suave (Soft Real Time), si la respuesta se produce después del tiempo límite es aceptable. La mayoría de los sistemas embebidos Implementan una mezcla de ambos comportamientos.

2) Kernel de FreeRTOS

a- Explique y desarrolle las características de una "Tarea" en FreeRTOS. Proponga un ejemplo de una tarea manteniendo la sintaxis correcta.

Una tarea es, en esencia, un pequeño programa independiente dentro de la aplicación, que el scheduler de FreeRTOS gestiona junto a las demás. Son como hilos de ejecución, cada una con su contexto.

Las tareas se implementan como funciones cuyo prototipo es siempre el mismo. Tienen un punto de entrada y normalmente corren en forma permanente dentro de un lazo infinito, sin retornar nunca. Una sola definición de función puede usarse para crear cualquier cantidad de tareas. Cada instancia creada es una ejecución separada, independiente de las demás. Cada instancia de tarea tiene su propia pila y su propia copia de las variables declaradas dentro de la función. Esto es así *solo si las variables son locales*; si se declaran como "static" existirá una única copia compartida entre todas las instancias.

Permite pasarle a cada instancia un dato distinto al momento de su creación (por ejemplo, una cadena a imprimir), evitando duplicar código para tareas casi idénticas. También tienen definidas una prioridad que utiliza el scheduler para ordenar la ejecución. Y pudimos ver que también pueden transicionar entre estados "Running, Ready, Blocked y Suspended", esto es justamente lo que distingue a una tarea de una función común.

Ejemplo de tarea:

```
//Tarea 1

void Tarea1 ( void *pvParameters )
{
    const char *pcTaskName = "Tarea 1 en ejecucion.\r\n";

    volatile unsigned long ul;

    /* Como en la mayoría de las tareas, se implementa en un lazo infinito. */
    for( ; ; )
```

```

    {

/* Imprime el nombre de la tarea. */

vPrintString ( pcTaskName );

/* Retarda un periodo. */

for ( ul=0; ul < mainDELAY_LOOP_COUNT; ul++)

    {

        }

    }

}

void Tarea2 ( void *pvParameters )

{

const char *pcTaskName = "Ejecutando Tarea2\r\n";

volatile unsigned long ul;

/* lazo infinito. */

for( ; ; )

{

/* Imprime el nombre de la tarea. */

vPrintString ( pcTaskName );

/* Retarda un periodo. */

for ( ul=0; ul < mainDELAY_LOOP_COUNT; ul++)

    {

        }

    }

}

int main ( void )

{

xTaskCreate ( Tarea1, "Tarea1", 1000, NULL, 1, NULL );

xTaskCreate ( Tarea2, "Tarea2", 1000, NULL, 1, NULL );

```

```
vTaskStartScheduler();
```

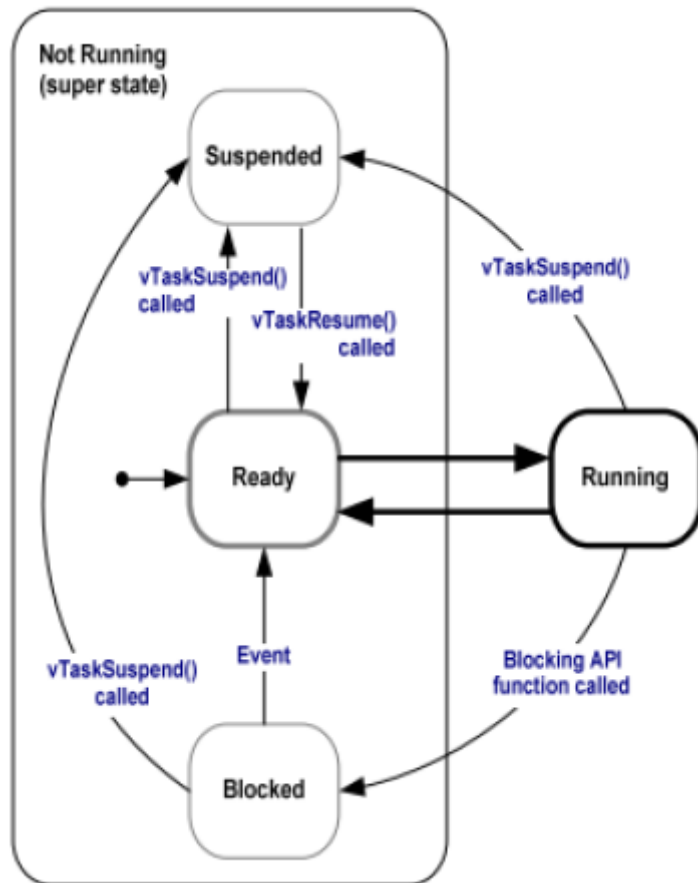
```
for ( ; ; )
```

```
}
```

b-Explique y desarrolle los “Estados de una Tarea” y sus transiciones. Indique las funciones que permiten las transiciones.

Toda tarea que no se encuentra ejecutándose está en estado de “No Ejecución o Not Running”, mientras que la tarea que si se está ejecutando se encuentran en estado “Ejecución o Running”. Solo una tarea puede ejecutarse a la vez y como vimos, el Scheduler elige siempre ejecutar la tarea de mayor prioridad que esté lista para ejecutarse.

Las tareas, a diferencia de las funciones tienen subestados que son controlados por el Scheduler o planificador. Entre los diferentes subestados se encuentran los siguientes: Ready (Lista para ejecutarse), Suspend (Suspendido), Resume (Reactivado) y Bloqued (Bloqueado). Entonces estas tareas van transicionando entre estos subestados según el siguiente diagrama.



Como vemos en el diagrama, el estado Not Running es un superestado que contiene los diferentes subestados: Blocked, Suspended y Ready. Mientras que cuando la tarea pasa al estado de Ejecución está en “Running”.

Cuando inicializamos una tarea, esta pasa al subestado Ready (lista para ejecutarse). El Scheduler va a ejecutar solo una tarea a la vez y seleccionara siempre la tarea de mayor prioridad. Al ejecutar una tarea, esta pasa al estado Running. En este estado de ejecución, la tarea tiene dos opciones de pasar al estado “Not Running”, transicionando al subestado Suspended o Blocked.

Se pasa al estado Suspended cuando se quiere Suspender intencionalmente la tarea desde el programa y se la puede hacer llamando a la función API `vTaskSuspend()`. En este caso la tarea ya no queda disponible para que el scheduler la vuelva a ejecutar hasta que se la saque de ese subestado y se la pase a Ready mediante el llamado a la función API `vTaskResume()` o `vTaskResumeFromISR()`. Una tarea en estado Ready también puede ser suspendida llamando a la misma función API `vTaskResume()`.

Una tarea que pasa del estado Running al subestado Bloqued queda en ese estado esperando un evento del sistema. Esos eventos pueden ser temporales o de sincronización (como el caso de una tarea que espere una cola por ejemplo). En ese caso, la tarea saldrá del estado de Bloqueo y pasará a Ready y Running finalmente.

Desde el subestado de Bloqueo, también se puede pasar a estado suspendido llamando a la función API `vTaskSuspend()`. Luego si llamáramos a la función API `vTaskResume()`, esta tarea quedaría nuevamente en estado Ready, lista para ejecutarse. También, desde el estado Blocked FreeRTOS puede desbloquearla si ocurre un evento como por ejemplo: Una tarea que este esperando bloqueada que una cola tenga un dato. Cuando la cola deje de estar vacía se desbloqueará dicha tarea pasando a Ready y luego a Running.

c-¿A qué nos referimos con el concepto de “tareas de procesamiento continuo”? ¿y a que nos referimos con el concepto de “tareas manejadas por eventos”?

Las tareas de procesamiento continuo son las que nunca llaman a funciones API que las bloqueen, por lo que siempre están en estado Running o Ready, nunca en Blocked. Constantemente tienen algo para hacer, aunque ese "algo" sea trivial.

El concepto de tareas manejadas por evento se refiere a aquellas que se quedan en estado bloqueado sin consumir procesamiento hasta que el evento esperado ocurra. Con esto se logra que el scheduler pueda ejecutar otras tareas en Ready optimizando los recursos.

d-¿Cuáles son los tipos de eventos por los que una tarea puede estar esperando al entrar en el estado bloqueado? Explicar.

Cuando una tarea pasa al estado Blocked, queda esperando a que ocurra un evento. Hay dos tipos de eventos que pueden sacarla de ese estado:

- Eventos temporales: Es cuando se programa una espera de tiempo (un retardo) en valores absolutos, como por ejemplo 100ms.

- Eventos de Sincronización: Este tipo de evento es generado desde una tarea o interrupción. Como por ejemplo, una tarea que se bloquee esperando que ingrese un dato a una cola o esperando un semáforo.

e- ¿Cuál es la función de la IDLE TASK? ¿Puede el procesador no estar ejecutando ninguna tarea en algún momento? Explicar.

El procesador siempre necesita algo para ejecutar. Siempre debe haber al menos una tarea en ejecución. Para garantizar esto, FreeRTOS crea automáticamente la tarea ociosa cuando se llama a la función API `vTaskStartScheduler()`. El programador no la crea explícitamente; el kernel se encarga de ello. La idle task convierte esta limitación en una herramienta útil: en vez de desperdiciar ese tiempo, FreeRTOS lo aprovecha para limpieza de memoria, métricas de uso de CPU, o ahorro de energía.

3) Desarrolle como es el almacenamiento de datos en el recurso “Cola” proporcionado por el Kernel.

Una cola es el mecanismo principal de comunicación y sincronización entre tareas en FreeRTOS. Permite transferir datos de forma segura en un entorno multitarea, sin necesidad de variables globales compartidas ni secciones críticas manuales. Funciona como un buffer de datos del tipo FIFO (First In , First Out). La longitud de la cola es la cantidad de elementos que contiene. Y cuando se crea la cola se define el tipo de dato que va a ser cada elemento (int,char,struct,etc).

Las colas son creadas llamando a la función API:

```
QueueHandle_t xQueueCreate( UBaseType_t uxQueueLength, UBaseType_t uxItemSize );
```

Donde:

uxQueueLength indica la longitud de cola

uxItemSize: Indica el tamaño de cada elemento.

Los estados posibles de la cola son los siguientes: Vacía (sin datos) , Con Datos y Llena. Para el caso de Cola Vacía o Llena, se notifica con un código de error para pasar las tareas que Reciben o insertan dato estado bloqueado.

Los datos pueden ser insertados en la cola llamando a la función API xQueueSend que inserta los datos al final de la cola. O se puede insertar el dato como primer elemento de la cola con la función API xQueueSendToFront.

Llamando a la función API xQueueReceive se pueden leer datos de la cola, del elemento del frente según dijimos trabaja en modo FIFO.

4) Explique utilizando un ejemplo con código el porqué es común que el recurso Cola posea múltiples tareas escritoras y una sola tarea lectora. Explique el proceso de bloqueo por escritura y bloqueo por lectura en el recurso Cola.

En sistemas embebidos como en este caso, es muy frecuente tener múltiples fuentes de datos (sensores, periféricos, interrupciones) que deben reportar información a una única tarea central que la procesa, toma decisiones o la envía por red. Del lado de los escritores cada fuente de datos es independiente, opera a su propia frecuencia y no necesita saber qué hacen las demás. Solo "deposita" su dato en la cola y sigue trabajando. Múltiples escritores no generan conflictos entre sí porque el kernel gestiona el acceso al buffer interno de forma atómica , entonces una operación de escritura completa sus sizeof(xData) bytes antes de que otra tarea pueda escribir.

Un ejemplo de un caso de múltiples escritores con un lector se muestra a continuación: En este caso, los escritores trabajan con una prioridad mayor al lector.

// Definición del tipo de elemento de cola

typedef struct

```
{
    unsigned char ucValue; /* El dato */
    unsigned char ucSource; /* Quién lo envió */
} xData;
```

// Dato a enviar por los dos escritores

static const xData xStructsToSend[2] =

```
{
    { 200, Emisor_1 },
    { 400, Emisor_2 }
};
```

};

static void vSenderTask(void *pvParameters)

```
{
    portBASE_TYPE xStatus;

    const portTickType xTicksToWait = 100 / portTICK_RATE_MS;
```

```

for( ;; )
{

    xStatus = xQueueSendToBack( xQueue,
                                pvParameters,
                                xTicksToWait );

    if( xStatus != pdPASS )
    {
        vPrintString( " No se pudo enviar.\r\n" );
    }

    taskYIELD(); //Sede el CPU
}
}

//Tarea lectora
static void vReceiverTask( void *pvParameters )
{
    xData xReceivedStructure;
    portBASE_TYPE xStatus;

    for( ;; )
    {
        if( uxQueueMessagesWaiting( xQueue ) != 3 )
        {
            vPrintString( "La cola esta llena.\n" );
        }

        /* Lee un ítem*/
        xStatus = xQueueReceive( xQueue,
                                &xReceivedStructure,
                                0 );

        if( xStatus == pdPASS )
        {

```

```

        if( xReceivedStructure.ucSource == Emisor_1 )
        {
            vPrintStringAndNumber( " Emisor_1 = ",
                                   xReceivedStructure.ucValue );
        }
        else
        {
            vPrintStringAndNumber( " Emisor_2 = ",
                                   xReceivedStructure.ucValue );
        }
    }
    else
    {
        vPrintString( "No se pudo recibir el dato.\r\n" );
    }
}
}

```

```

xQueueHandle xQueue;

```

```

int main( void )

```

```

{
    /* Cola para 3 estructuras de tipo xData */
    xQueue = xQueueCreate( 3, sizeof( xData ) );

    if( xQueue != NULL )
    {
        /* Dos escritores con prioridad 2 (MAYOR).
           Cada uno recibe su estructura como parámetro. */
        xTaskCreate( vSenderTask, "Sender1", 1000,
                    &( xStructsToSend[0] ), 2, NULL );
        xTaskCreate( vSenderTask, "Sender2", 1000,
                    &( xStructsToSend[1] ), 2, NULL );

        /* Un lector con prioridad 1 (MENOR) */
        xTaskCreate( vReceiverTask, "Receiver", 1000,

```

```
NULL, 1, NULL );
```

```
vTaskStartScheduler();  
  
}  
  
for( ;; );  
  
}
```

5) A la hora de enviar datos compuestos: explique cómo se lleva a cabo dicho proceso y por qué no genera conflictos con la definición de la macro del recurso. ¿Cuál sería la implementación si los datos fueran “grandes”, lo que ocasionaría un problema con la memoria RAM? (Pag. 91)

Los datos compuestos, son datos que agrupan múltiples campos relacionados en una única estructura. Y dado que en la práctica es común para una tarea recibir datos de diferentes fuentes a una cola, generalmente es necesario saber de qué fuente proviene el dato para poder procesarlo adecuadamente. Una forma de lograrlo es creando datos del tipo estructura donde tenemos un campo dato y otro que identifique la fuente de donde proviene. Esto no genera conflictos con la definición de la macro ya que la escritura de datos a una cola se lleva a cabo copiando byte por byte de los datos a ser almacenados en la cola. La cola no guarda punteros a la estructura original, guarda una copia completa. La variable original puede destruirse, modificarse o salir de scope sin afectar el dato ya encolado.

No hay conflicto con el tamaño definido en `xQueueCreate()` porque en la creación se especificó exactamente `sizeof(xData)`. El kernel sabe que cada slot del buffer interno mide exactamente ese número de bytes, y copia exactamente esa cantidad en cada operación de escritura y lectura.

6) Explicar el funcionamiento del esquema de la fig 1:

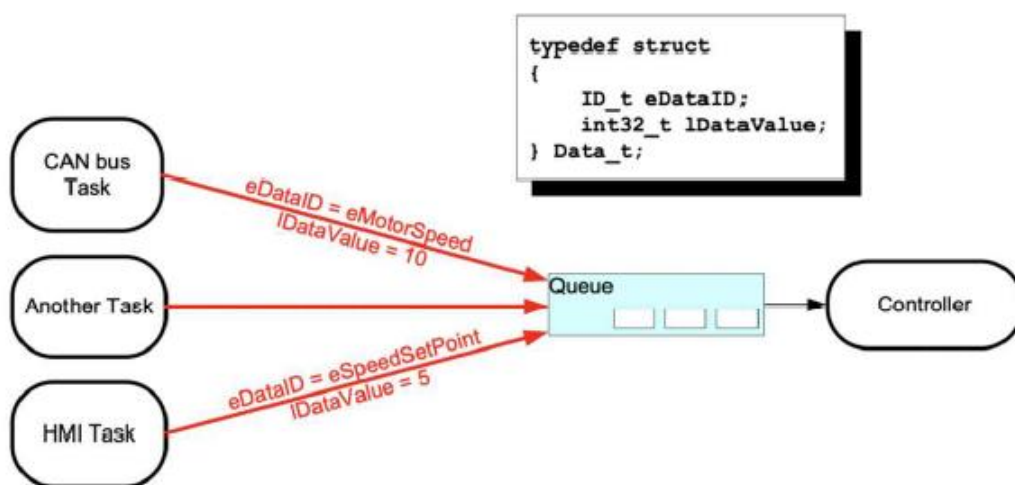


Fig. 1. Recibir datos de múltiples fuentes

La figura muestra exactamente el patrón múltiples escritores / un solo lector usando una cola que transfiere datos compuestos mediante estructuras. Usando colas para transferir tipos compuestos. Con dos campos en la misma estructura, el receptor recibe en un único ítem de cola tanto el dato como su significado, sin necesidad de múltiples colas ni variables globales.

CAN bus Task representa una tarea que lee el bus CAN del sistema (por ejemplo, de un variador de frecuencia o un motor) y cuando recibe un mensaje decodificado de velocidad de motor, arma una estructura `Data_t` con `eDataID = eMotorSpeed` y `lDataValue = 10` (la velocidad medida) y la encola.

Another Task también envía datos a la misma cola. En la figura su etiqueta no especifica qué tipo de dato envía, pero comparte la misma cola y la misma estructura.

HMI Task representa la interfaz hombre máquina (panel de operador, pantalla táctil, botones). Cuando el operador ingresa un nuevo punto de ajuste de velocidad, la tarea HMI arma una estructura con eDataID = eSpeedSetPoint y IDDataValue = 5 y la encola.

7) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Semáforos Mutex”.

Programa de ejemplo con Mutex.

```
#include "FreeRTOS.h"
```

```
#include "task.h"
```

```
#include "semphr.h"
```

```
#include <stdio.h>
```

```
// Declaración del Mutex
```

```
SemaphoreHandle_t xMutex;
```

```
// Recurso compartido (ejemplo: contador)
```

```
int contador = 0;
```

```
// Tarea A
```

```
void TareaA(void *pvParameters) {
```

```
    for (;;) {
```

```
        // Intentar tomar el mutex
```

```
        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE) {
```

```
            // Sección crítica
```

```
            contador++;
```

```
            printf("Tarea A incrementa contador: %d\n", contador);
```

```
        // Liberar el mutex
```

```
        xSemaphoreGive(xMutex);
```

```
    }
```

```
    vTaskDelay(pdMS_TO_TICKS(500));
```

```
}
```

```
}
```

```
// Tarea B
```

```
void TareaB(void *pvParameters) {
```

```

for (;;) {

    if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE) {

        contador += 2;

        printf("Tarea B incrementa contador: %d\n", contador);

        xSemaphoreGive(xMutex);

    }

    vTaskDelay(pdMS_TO_TICKS(700));

}

}

int main(void) {

    // Crear el mutex

    xMutex = xSemaphoreCreateMutex();

    if (xMutex != NULL) {

        // Crear tareas

        xTaskCreate(TareaA, " TareaA", 1000, NULL, 1, NULL); //Con prioridad 1

        xTaskCreate(TareaB, " TareaB", 1000, NULL, 1, NULL); //Con prioridad 1

        // Iniciar el scheduler

        vTaskStartScheduler();

    }

    for (;;)

}

```

8) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Inversión de Prioridad”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

```

#include "FreeRTOS.h"

#include "task.h"

#include "queue.h"

#include "semphr.h"

#include <stdio.h>

```

```

QueueHandle_t xQueue;

```

```
SemaphoreHandle_t ElMutex;
```

```
// Tarea de baja prioridad que simula acceso a un recurso
```

```
void TareaBajaPrioridad (void *pvParameters) {  
    for (;;) {  
        if (xSemaphoreTake(ElMutex, portMAX_DELAY) == pdTRUE) {  
            // Accede al recurso  
            printf("Tarea Baja: usando recurso...\n");  
  
            // Simula trabajo prolongado  
            vTaskDelay(pdMS_TO_TICKS(2000));  
  
            // Libera el recurso  
            xSemaphoreGive(xMutex);  
            printf("Tarea Baja: recurso liberado.\n");  
        }  
        vTaskDelay(pdMS_TO_TICKS(500));  
    }  
}
```

```
// Tarea de alta prioridad (necesita el recurso rápidamente)
```

```
void TareaAltaPrioridad (void *pvParameters) {  
    for (;;) {  
        if (xSemaphoreTake(ElMutex, portMAX_DELAY) == pdTRUE) {  
            printf("Tarea Alta: accediendo al recurso.\n");  
  
            // Envía un mensaje a la cola  
            int dato = 42;  
            xQueueSend(xQueue, &dato, portMAX_DELAY);  
  
            xSemaphoreGive(ElMutex);  
            printf("Tarea Alta: recurso liberado.\n");  
        }  
        vTaskDelay(pdMS_TO_TICKS(1000));  
    }  
}
```

```

}

// Tarea de prioridad intermedia (no usa el recurso, pero consume CPU)
void TareaMediaPrioridad (void *pvParameters) {
    for (;;) {
        printf("Tarea Media: ejecutando trabajo...\n");
        vTaskDelay(pdMS_TO_TICKS(300));
    }
}

int main(void) {
    // Crear cola
    xQueue = xQueueCreate(5, sizeof(int));

    // Creamos mutex
    ElMutex = xSemaphoreCreateMutex();

    if (xQueue != NULL && ElMutex != NULL) {
        // Creamos tareas con diferentes prioridades
        xTaskCreate(TareaBajaPrioridad, " TareaBajaPrioridad ", 1000, NULL, 1, NULL);
        xTaskCreate(TareaMediaPrioridad, " TareaMediaPrioridad ", 1000, NULL, 2, NULL);
        xTaskCreate(TareaAltaPrioridad, " TareaAltaPrioridad ", 1000, NULL, 3, NULL);

        // Iniciar scheduler
        vTaskStartScheduler();
    }

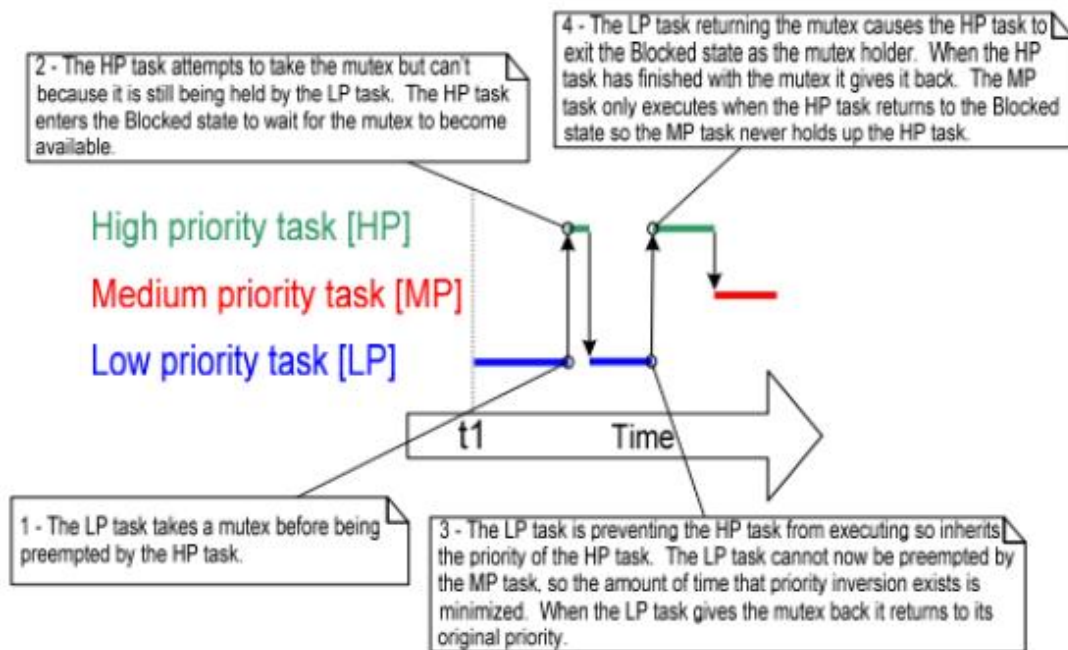
    for (;;)
}

```

9) Explicar el concepto de “Herencia de Prioridad” utilizando diagramas gráficos que permitan comprenderlo.

La principal distinción entre un mutex y un semáforo binario radica en que el mutex incorpora de forma automática un mecanismo de herencia de prioridad. Este mecanismo permite que, cuando una tarea de baja prioridad posee el mutex y una tarea de mayor prioridad intenta obtenerlo, la primera aumente temporalmente su prioridad al nivel de la segunda. De este modo, la tarea que tiene el mutex “hereda” la prioridad de la tarea bloqueada, garantizando que

libere el recurso lo antes posible y evitando demoras innecesarias en la ejecución de tareas críticas.



10) Desarrollar un programa ejemplo que permita comprender el concepto de “Punto Muerto”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

```
#include "FreeRTOS.h"
```

```
#include "task.h"
```

```
#include "queue.h"
```

```
#include "semphr.h"
```

```
#include <stdio.h>
```

```
SemaphoreHandle_t ElMutexA;
```

```
SemaphoreHandle_t ElMutexB;
```

```
QueueHandle_t LaQueue;
```

```
// Tarea 1: toma MutexA y luego intenta MutexB
```

```
void Tarea1(void *pvParameters) {
```

```
    for (;;) {
```

```
        if (xSemaphoreTake(ElMutexA, portMAX_DELAY) == pdTRUE) {
```

```
            printf("Tarea 1: tomó MutexA\n");
```

```
            vTaskDelay(pdMS_TO_TICKS(100)); // Simula trabajo
```

```
            if (xSemaphoreTake(ElMutexB, portMAX_DELAY) == pdTRUE) {
```

```

        printf("Tarea 1: tomó MutexB\n");

        int dato = 1;

        xQueueSend(LaQueue, &dato, portMAX_DELAY);

        xSemaphoreGive(ElMutexB);
    }

    xSemaphoreGive(ElMutexA);
}

vTaskDelay(pdMS_TO_TICKS(500));
}
}

// Tarea 2: toma MutexB y luego intenta MutexA
void Tarea2(void *pvParameters) {
    for (;;) {
        if (xSemaphoreTake(ElMutexB, portMAX_DELAY) == pdTRUE) {
            printf("Tarea 2: tomó MutexB\n");

            vTaskDelay(pdMS_TO_TICKS(100)); // Simula trabajo

            if (xSemaphoreTake(ElMutexA, portMAX_DELAY) == pdTRUE) {
                printf("Tarea 2: tomó MutexA\n");

                int dato = 2;

                xQueueSend(LaQueue, &dato, portMAX_DELAY);

                xSemaphoreGive(ElMutexA);
            }

            xSemaphoreGive(ElMutexB);
        }

        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

int main(void) {
    ElMutexA = xSemaphoreCreateMutex();

    ElMutexB = xSemaphoreCreateMutex();

```

```
LaQueue = xQueueCreate(5, sizeof(int));
```

```
if (ElMutexA != NULL && ElMutexB != NULL && LaQueue != NULL) {
```

```
    xTaskCreate(Tarea1, "Tarea1", 1000, NULL, 2, NULL);
```

```
    xTaskCreate(Tarea2, "Tarea2", 1000, NULL, 2, NULL);
```

```
    vTaskStartScheduler();
```

```
}
```

```
for (;;);
```

```
}
```